



UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO DO UÍGE
CURSO DE ENGENHARIA INFORMÁTICA

Trabalho em Grupo de LINGUAGEM DE PROGRAMAÇÃO VI

Sistema Hospitalar Inteligente

Por:

1. Filipe Miguel João
2. Filipe Pedro Muanza
3. Guiola André
4. Nsimba Alberto Samuel
5. Pedro Marcos Kudikusa

3º ano
Turma: TL 101
Período: Manhã

ORIENTADOR

Moyo Kanivengidio, Msc.

Uíge, Maio de 2026

ÍNDICE

0. INTRODUÇÃO	1
1. FUNCIONALIDADES PRINCIPAIS.....	2
2. ARQUITECTURA DO SISTEMA	2
3. DIAGRAMA DE CLASSES UML.....	3
3.1. Relações entre Entidades da Base de Dados	3
4. TECNOLOGIAS UTILIZADAS	4
5. MÓDULO DE INTELIGÊNCIA ARTIFICIAL	4
5.1. Algoritmo: RandomForestClassifier.....	4
5.2. Features de Entrada (10 variáveis)	4
6. BASE DE DADOS E MODELO RELACIONAL	5
7. INTERFACE GRÁFICA (GUI)	6
CONCLUSÃO	7
REFERÊNCIAS BIBLIOGRÁFICAS	8

0. INTRODUÇÃO

O presente relatório documenta o desenvolvimento do Sistema de Gestão Hospitalar Inteligente, denominado Hospital Kivi, no âmbito da disciplina de Linguagem de Programação VI (Python). O projecto visa construir uma aplicação desktop completa, integrando Programação Orientada a Objectos avançada, interface gráfica profissional, persistência relacional de dados e um módulo de Inteligência Artificial para triagem automática de pacientes.

O problema central abordado é a ineficiência nos processos de triagem hospitalar em contextos com recursos limitados. A solução proposta automatiza a classificação de prioridade de atendimento com base nos sintomas do paciente, auxiliando profissionais de saúde na tomada de decisão rápida e fundamentada.

1. FUNCIONALIDADES PRINCIPAIS

- Autenticação de utilizadores com controlo de papéis (*Admin, Médico, Recepcionista, Enfermeiro*).
- Gestão completa de pacientes (CRUD com pesquisa avançada).
- Agendamento e gestão de consultas médicas.
- Triagem automática com modelo RandomForest (scikit-learn), classificação em 5 níveis de prioridade.
- Prontuário médico electrónico por paciente.
- Dashboard analítico com gráficos (Matplotlib).
- Gestão de utilizadores do sistema.
- Persistência em SQLite via SQLAlchemy ORM.

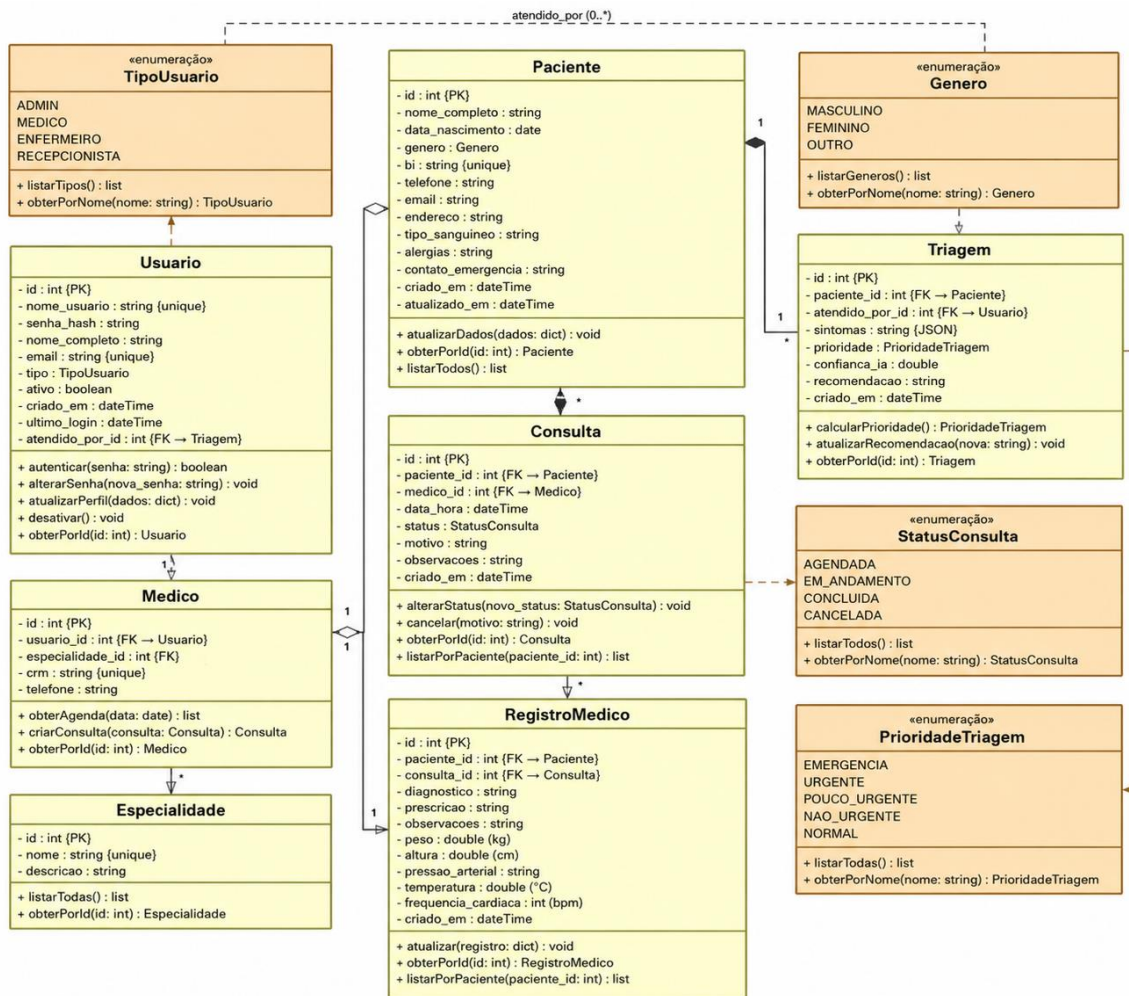
2. ARQUITECTURA DO SISTEMA

A aplicação segue a arquitectura em camadas MVC (Model-View-Controller) adaptada para aplicações desktop, com separação clara de responsabilidades:

Camada	Módulo	Responsabilidade
Model (Dados)	database/	Modelos SQLAlchemy, Repositórios DAO, Conexão à BD
Controller (Lógica)	services/ + core/	Serviços de negócio, Abstrações, Decoradores
View (Interface)	gui/	Janelas e Frames CustomTkinter
IA	ai/	Modelo RandomForest de triagem automática

3. DIAGRAMA DE CLASSES UML

Para o Diagrama de Classe, utilizamos o Astah, sendo uma ferramenta extremamente poderosa na criação de Diagramas de Classe UML.



3.1. Relações entre Entidades da Base de Dados

Entidade	Relações
Usuario	1:1 Medico (perfil médico)
Medico	N:1 Especialidade 1:N Consulta
Paciente	1:N Consulta 1:N RegistroMedico 1:N Triagem
Consulta	N:1 Paciente N:1 Medico 1:1 RegistroMedico
Triagem	N:1 Paciente N:1 Usuario
RegistroMedico	N:1 Paciente N:1 Consulta

4. TECNOLOGIAS UTILIZADAS

Tecnologia	Versão	Finalidade
Python	3.10+	Linguagem principal de desenvolvimento
CustomTkinter	5.2.2	Interface gráfica moderna (GUI)
SQLAlchemy	2.0.30	ORM e abstracção de base de dados
SQLite	Nativa	Sistema de Gestão de Base de Dados
scikit-learn	1.4.2	Modelo de IA (RandomForestClassifier)
NumPy	1.26.4	Computação numérica para o modelo IA
Matplotlib	3.9.0	Gráficos e visualizações no dashboard
Pillow	10.3.0	Processamento de imagens
ReportLab	4.x	Geração deste relatório em PDF

5. MÓDULO DE INTELIGÊNCIA ARTIFICIAL

Problema e Abordagem

O módulo de IA implementa um classificador supervisionado para a triagem automática de pacientes, inspirado no protocolo de Manchester, o modelo classifica o paciente em uma de 5 prioridades: ***Emergência, Urgente, Pouco Urgente, Não Urgente e Normal.***

5.1. Algoritmo: RandomForestClassifier.

Foi escolhido o algoritmo Random Forest (Floresta Aleatória) da biblioteca scikit-learn pelos seguintes motivos:

- Robusto a outliers e dados ruidosos (comum em contextos clínicos).
- Não requer normalização extensiva das features.
- Fornece importância de features para interpretabilidade.
- Excelente desempenho com dados tabulares de dimensão moderada.
- Suporte nativo a classes desbalanceadas via `class_weight='balanced'`.

5.2. Features de Entrada (10 variáveis)

Feature	Tipo	Escala	Descrição Clínica
---------	------	--------	-------------------

fever	Int	0–2	Febre: 0=Não, 1=Baixa (<38.5), 2=Alta (>=38.5)
pain_level	Int	0–10	Escala numérica de dor (EVA)
breathing_diff	Int	0–2	Dispneia: 0=Nenhuma, 1=Leve, 2=Grave
consciousness	Int	0–2	Escala AVPU simplificada
bleeding	Int	0–2	Hemorragia: 0=Não, 1=Leve, 2=Grave
vomiting	Int	0–1	Presença de vômitos
chest_pain	Int	0–1	Dor torácica (risco cardíaco)
heart_rate_abnormal	Int	0–1	FC anormal (<60 ou >100 bpm)
age_group	Int	0–2	0=Criança, 1=Adulto, 2=Idoso
duration_hours	Int	0–72	Horas desde o início dos sintomas

6. BASE DE DADOS E MODELO RELACIONAL

- O sistema utiliza SQLite como SGBD e SQLAlchemy 2.0 como ORM. O ficheiro hospital.db é criado automaticamente na primeira execução. São implementadas 7 tabelas:
- users — Utilizadores do sistema com papéis e autenticação
- specialties — Especialidades médicas disponíveis
- doctors — Perfil médico (extensão de users)
- patients — Dados demográficos e clínicos dos pacientes
- appointments — Consultas agendadas, em curso ou concluídas
- medical_records — Prontuário médico por consulta (sinais vitais, diagnóstico)
- triages — Resultados de triagem IA com prioridade e confiança
- Padrão de Acesso a Dados (DAO/Repository)

Todas as operações de persistência passam pela camada de repositório. O SQLAlchemyRepository genérico implementa CRUD completo; repositórios especializados adicionam consultas de domínio (e.g., PatientRepository.search(), AppointmentRepository.count_by_status()).

7. INTERFACE GRÁFICA (GUI)

- A GUI foi desenvolvida com CustomTkinter 5.2.2, adoptando um tema escuro moderno (inspirado no GitHub Dark). A navegação é feita por uma barra lateral persistente com acesso a 6 módulos:
- Dashboard: KPIs em tempo real, gráfico de pizza (consultas por status) e barras horizontais (triagens por prioridade).
- Pacientes: Tabela com pesquisa em tempo real, formulário de criação/edição com scrollable frame.
- Consultas: Agendamento, filtro por status, conclusão/cancelamento e acesso ao prontuário.
- Triagem IA: Formulário de sintomas com sliders intuitivos, execução assíncrona da IA, barra de confiança e histórico.
- Prontuários: Vista de dois painéis: lista de pacientes e registos clínicos com detalhes ao clicar.
- Utilizadores: Gestão de contas (apenas para Admin) e alteração de palavra-passe.

CONCLUSÃO

O Sistema de Gestão Hospitalar Kivi atingiu os objectivos propostos no enunciado do trabalho prático. A integração de POO Avançada (classes abstractas, polimorfismo, decoradores, context managers), GUI profissional (CustomTkinter com tema escuro), persistência robusta (SQLAlchemy + SQLite) e Inteligência Artificial (RandomForest para triagem) demonstra uma solução tecnicamente sólida e com valor real para o domínio hospitalar.

O sistema é facilmente extensível, novos módulos (farmácia, laboratório, faturação) podem ser adicionados seguindo os padrões de arquitectura estabelecidos. O modelo de IA pode ser melhorado com dados clínicos reais e técnicas de aprendizagem activa.

REFERÊNCIAS BIBLIOGRÁFICAS

Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.

SQLAlchemy Documentation (2024). ORM Quick Start. <https://docs.sqlalchemy.org>

scikit-learn Developers (2024). RandomForestClassifier API. <https://scikit-learn.org>

CustomTkinter Documentation (2024). <https://customtkinter.tomschimansky.com>

Matthes, E. (2023). *Python Crash Course, 3rd Ed.* No Starch Press.

Raschka, S. & Mirjalili, V. (2019). *Python Machine Learning, 3rd Ed.* Packt.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

PEP 8 — Style Guide for Python Code. <https://peps.python.org/pep-0008/>

Manchester Triage System. <https://www.triagenet.net>